

Escuela de Computación
Ingeniería en Computación
IC6600 - Principio de Sistemas Operativos

Proyecto 2

Simulación de algoritmos de paginación

Estefanía Delgado Castillo
tefadc@estudiantec.cr
2018112159

Jorge Esteban Benavides Castro
jorbenavides@estudiantec.cr
2022230697

San José, Costa Rica
Mayo 2026

Índice general

1. Introducción	2
2. Desarrollo	3
2.1. Algoritmo Óptimo (OPT)	3
2.1.1. Descripción teórica	3
2.1.2. Implementación en el proyecto	3
2.2. Algoritmos FIFO y Second Chance	5
2.2.1. Descripción teórica	5
2.2.2. Implementación en el proyecto	5
2.3. Algoritmo LRU	6
2.3.1. Descripción teórica	6
2.3.2. Implementación en el proyecto	6
2.4. Algoritmo LFU	7
2.4.1. Descripción teórica	7
2.4.2. Implementación en el proyecto	7
2.5. Estado general de la implementación	8
3. Enlace al código e instrucciones	9
3.1. Instrucciones de compilación y ejecución	9
4. Conclusiones	10

Capítulo 1

Introducción

La gestión de memoria es una de las funciones fundamentales de los sistemas operativos modernos (Silberschatz et al., 2018). Consiste en administrar la jerarquía de memoria (registros, caché, RAM y disco) de forma eficiente, asignando espacio a los procesos y garantizando que puedan ejecutarse incluso si la memoria principal es insuficiente. Cuando la RAM es limitada, se emplean técnicas como la **paginación** y la **memoria virtual** (Tanenbaum & Bos, 2015).

La paginación divide la memoria física en bloques de tamaño fijo llamados *marcos de página* (frames) y la memoria lógica de cada proceso en bloques del mismo tamaño llamados *páginas*. La memoria virtual extiende la RAM utilizando el disco duro como respaldo, permitiendo ejecutar procesos cuyo tamaño total supera la memoria física disponible. El hardware (MMU) traduce direcciones virtuales a físicas y, cuando una página no está en RAM, se produce un *fallo de página* que provoca su carga desde el disco (Silberschatz et al., 2018).

Para decidir qué página reemplazar cuando la RAM está llena, se utilizan distintos **algoritmos de reemplazo de páginas**. Este proyecto implementa una simulación que compara el comportamiento del algoritmo óptimo (OPT) con cuatro algoritmos prácticos: FIFO, Second Chance (SC), LRU y LFU. La simulación permite visualizar métricas como tiempo total, porcentaje de thrashing, fragmentación interna y uso de memoria real/virtual, siguiendo las especificaciones del curso.

Capítulo 2

Desarrollo

2.1. Algoritmo Óptimo (OPT)

2.1.1. Descripción teórica

El algoritmo óptimo (OPT, también llamado MIN o Belady) reemplaza la página que no será utilizada durante el mayor tiempo en el futuro, o que nunca volverá a usarse (Silberschatz et al., 2018). Es imposible de implementar en un sistema real porque requiere conocer el futuro de las referencias, pero se utiliza como punto de referencia teórico para evaluar otros algoritmos.

2.1.2. Implementación en el proyecto

En el archivo `algorithms.js` se define la función `optimal(mmu, opIndex)`. Dicha función evalúa cada página presente en la RAM y busca, en la lista de operaciones futuras (`mmu.operations`), la próxima vez que se usará esa misma página (comparando `op.type === "use"` y `op.ptr === page.ptr`). Si no se encuentra un uso futuro, se asigna una prioridad infinita (se elige esa página como víctima inmediata). El código real es:

```
1 function optimal(mmu, opIndex) {
2   ensureOptimalFutureUses(mmu);
3
4   let victimIndex = 0;
5   let farthestUse = -1;
6
7   for (let i = 0; i < mmu.ram.length; i++) {
8     const page = mmu.ram[i];
9     const nextUse = nextUseOfPtr(mmu, page.ptr, opIndex);
10
11     if (nextUse === Infinity) return i;
12
13     if (nextUse > farthestUse) {
14       farthestUse = nextUse;
15       victimIndex = i;
16     }
17   }
18 }
```

```

17     }
18
19     return victimIndex;
20 }
21
22 function ensureOptimalFutureUses(mmu) {
23     if (mmu.optFutureUses) return;
24
25     mmu.optFutureUses = new Map();
26     mmu.optFutureUseCursor = new Map();
27
28     for (let i = 0; i < mmu.operations.length; i++) {
29         const op = mmu.operations[i];
30         if (op.type !== "use") continue;
31
32         if (!mmu.optFutureUses.has(op.ptr)) {
33             mmu.optFutureUses.set(op.ptr, []);
34             mmu.optFutureUseCursor.set(op.ptr, 0);
35         }
36
37         mmu.optFutureUses.get(op.ptr).push(i);
38     }
39 }
40
41 function nextUseOfPtr(mmu, ptr, opIndex) {
42     const uses = mmu.optFutureUses.get(ptr);
43     if (!uses) return Infinity;
44
45     let cursor = mmu.optFutureUseCursor.get(ptr) ?? 0;
46
47     while (cursor < uses.length && uses[cursor] <= opIndex) {
48         cursor++;
49     }
50
51     mmu.optFutureUseCursor.set(ptr, cursor);
52     return cursor < uses.length ? uses[cursor] : Infinity;
53 }

```

Processes		Sim-Time	
0		4381s	
RAM KB	RAM %	V-RAM KB	V-RAM %
0	0%	0	0%
Pages	Thrashing	Thrashing %	Fragmentation
0	3800s	87%	0B

Figura 2.1: Algo. Opt. con P=122 y N=500

2.2. Algoritmos FIFO y Second Chance

2.2.1. Descripción teórica

- **FIFO (First In, First Out)** reemplaza la página que lleva más tiempo en memoria. Es simple pero sufre la "anomalía de Belady" (aumentar el número de marcos puede aumentar la tasa de fallos) (Tanenbaum & Bos, 2015).
- **Second Chance** mejora FIFO dando una segunda oportunidad a las páginas que han sido referenciadas recientemente. Se organiza como una cola circular con un bit de referencia; si el bit está a 1, se pone a 0 y se pasa a la siguiente página; si está a 0, se selecciona como víctima (Silberschatz et al., 2018).

2.2.2. Implementación en el proyecto

- **FIFO:** Implementado en el archivo `algorithms.js`. El algoritmo FIFO se implementó de forma sencilla, simplemente eligiendo la página que tenga el menor valor en `page.loadedAt`, ya que éste representa el momento en el que fue cargada al sistema.

```

1 export function pickVictim(mmu, opIndex) {
2   const pages = mmu.ram;
3   if (mmu.algorithm === "FIFO") return minIndex(pages, p => p.
      loadedAt);
4   //Otros codigos
5 }
6
7 const minIndex = (arr, score) => arr.reduce((best, p, i) =>
      score(p) < score(arr[best]) ? i : best, 0);

```

- **Second Chance:** Implementado en el archivo `algorithms.js`. Second Chance utiliza un puntero circular (`mmu.clockHand`) para recorrer las páginas del MMU, así mismo hace uso del un bit de referencia `page.ref`. Las páginas con `r-bit = 1` han sido accedidas recientemente. Cada vez que se accede a una página se modifica `page.ref` y se pone en 0. Si accede a una página que ya tenía `page.ref = 0` la retorna para su reemplazo.

```

1 function secondChance(mmu) {
2   while (true) {
3     const p = mmu.ram[mmu.clockHand];
4     if (p.ref === 0) {
5       const out = mmu.clockHand;
6       mmu.clockHand = (out + 1) % mmu.ram.length;
7       return out;
8     }
9
10    p.ref = 0;
11    mmu.clockHand = (mmu.clockHand + 1) % mmu.ram.length;
12  }
13 }

```

■ Capturas de pantalla

Processes		Sim-Time	
0		4381s	
RAM KB	RAM %	V-RAM KB	V-RAM %
0	0%	0	0%
Pages	Thrashing	Thrashing %	Fragmentation
0	3800s	87%	0B

Processes		Sim-Time	
0		5509s	
RAM KB	RAM %	V-RAM KB	V-RAM %
0	0%	0	0%
Pages	Thrashing	Thrashing %	Fragmentation
0	5210s	99%	0B

Figura 2.2: Algo. Opt. vs FIFO, con P=122 y N=500

Processes		Sim-Time	
0		4381s	
RAM KB	RAM %	V-RAM KB	V-RAM %
0	0%	0	0%
Pages	Thrashing	Thrashing %	Fragmentation
0	3800s	87%	0B

Processes		Sim-Time	
0		5293s	
RAM KB	RAM %	V-RAM KB	V-RAM %
0	0%	0	0%
Pages	Thrashing	Thrashing %	Fragmentation
0	4940s	93%	0B

Figura 2.3: Algo. Opt. vs SC, con P=122 y N=500

2.3. Algoritmo LRU

2.3.1. Descripción teórica

LRU (Least Recently Used) reemplaza la página que ha sido utilizada hace más tiempo. Se basa en el principio de localidad: es probable que una página usada recientemente vuelva a usarse pronto (Tanenbaum & Bos, 2015). Es costoso de implementar en hardware puro, pero en simulación se puede llevar un contador de último uso.

2.3.2. Implementación en el proyecto

Implementado en `algorithms.js`. Al igual que FIFO, LRU fue implementado de forma sencilla. Haciendo uso del atributo `lastUsed`, el cual se actualiza

en `MMU.use` cada vez que se accede a una página, asignándole el valor actual del reloj (`page.lastUsed = this.clock`). Para esto se hace uso de `minIndex` que obtiene el valor mínimo de `lastUsed` en la simulación. De esta forma, la página con el valor más pequeño es la menos recientemente usada.

```

1 export function pickVictim(mmu, opIndex) {
2   const pages = mmu.ram;
3   if (mmu.algorithm === "LRU") return minIndex(pages, p => p.
      lastUsed);
4   //Otros codigos
5 }
6
7 const minIndex = (arr, score) => arr.reduce((best, p, i) => score(p
  ) < score(arr[best]) ? i : best, 0);

```

Processes		Sim-Time	
0		4381s	
RAM KB	RAM %	V-RAM KB	V-RAM %
0	0%	0	0%
Pages	Thrashing	Thrashing %	Fragmentation
0	3800s	87%	0B

Processes		Sim-Time	
0		5281s	
RAM KB	RAM %	V-RAM KB	V-RAM %
0	0%	0	0%
Pages	Thrashing	Thrashing %	Fragmentation
0	4925s	53%	0B

Figura 2.4: Algo. Opt. vs LRU, con P=122 y N=500

2.4. Algoritmo LFU

2.4.1. Descripción teórica

LFU (Least Frequently Used) reemplaza la página que ha sido utilizada con menor frecuencia (menor número de accesos). Puede sufrir el problema de que páginas muy usadas al principio queden fijadas para siempre; por eso a veces se usan variantes con decaimiento de contadores (Silberschatz et al., 2018).

2.4.2. Implementación en el proyecto

Implementado en `algorithms.js`, y de forma similar a los anteriores LFU se implementa simplemente a través del atributo `freq`). Este atributo se incrementa en `MMU.use` cada vez que la página es accedida (`page.freq++`). Al elegir víctima, se selecciona la página con menor frecuencia de uso, a través de la función `minIndex`, para ser reemplazada.

```

1 export function pickVictim(mmu, opIndex) {
2   const pages = mmu.ram;
3   if (mmu.algorithm === "LFU") return minIndex(pages, p => p.freq);
4   //Otros codigos
5 }
6
7 const minIndex = (arr, score) => arr.reduce((best, p, i) => score(p
  ) < score(arr[best]) ? i : best, 0);

```


Processes		Sim-Time	
0		4229s	
RAM KB	RAM %	V-RAM KB	V-RAM %
0	0%	0	0%
Pages	Thrashing	Thrashing %	Fragmentation
0	3810s	90%	0B

Processes		Sim-Time	
0		4997s	
RAM KB	RAM %	V-RAM KB	V-RAM %
0	0%	0	0%
Pages	Thrashing	Thrashing %	Fragmentation
0	4770s	98%	0B

Figura 2.5: Algo. Opt. vs LFU, con P=122 y N=500

2.5. Estado general de la implementación

El punto de entrada del sistema es el archivo `index.html`, que proporciona la interfaz gráfica con la que interactúa el usuario. Desde esta interfaz se configuran parámetros como la cantidad de procesos, el número de operaciones, el algoritmo de reemplazo de páginas entre otros.

El control general del sistema recae sobre el archivo `main.js`. Este módulo se encarga de leer la configuración seleccionada por el usuario, generar o cargar la secuencia de instrucciones (según si se proporcionó o no) y crear la instancia principal del simulador. Así mismo, conecta los controles de la interfaz con las acciones de ejecución, pausa y avance paso a paso. La preparación de las instrucciones se realiza mediante `preparation.js`. Este componente permite generar secuencias, interpretar archivos de operaciones cargados por el usuario y convertirlas a un formato exportable.

Por otro lado, `simulation.js` coordina la ejecución de las instrucciones preparadas. Para cada operación, el simulador mantiene dos instancias independientes: el algoritmo óptimo (OPT) y otra que utiliza el algoritmo seleccionado.

La lógica principal está en `mmu.js`. En este archivo se busca simular el comportamiento de una Memory Management Unit (MMU). Cuando es necesario cargar una nueva página, la MMU solicita a los algoritmos de reemplazo, una página a ser removida. Los algoritmos de reemplazo están en `algorithms.js`. Acá se implementan los algoritmos FIFO, Second Chance, LRU, LFU y OPT.

Para representar la información del sistema, `models.js` define las estructuras como las páginas de memoria y los registros que relacionan punteros con sus páginas asociadas. Por su parte, `constants.js` centraliza los parámetros globales de la simulación, es decir, el tamaño de página, los marcos de memoria y los tiempos utilizados.

La visualización se da en conjunto con `index.html`, `styles.css` y `app.js`. Este último recibe ambas MMU y actualiza la interfaz cada vez que se ejecuta una operación.

Capítulo 3

Enlace al código e instrucciones

El código fuente completo se encuentra en el siguiente repositorio:

<https://github.com/AverageCastroEnjoyer/proyecto2>

3.1. Instrucciones de compilación y ejecución

Al tratarse de JavaScript, no requiere compilación. Para ejecutar la simulación solo hace falta ejecutar los siguientes comandos en terminal:

```
1 # Clonar el repositorio
2 git clone https://github.com/AverageCastroEnjoyer/proyecto2.
   git
3
4 # Moverse a la carpeta d nde se clonaron los archivos
5 cd proyecto2
6
7 # Inicializar el servidor donde se ejecutar la simulaci n
8 python3 -m http.server 8080
9
10 # Abrir un navegador y cargar el servidor
11 http://localhost:8080
12
13 # Una vez ah ya se pueden setear los p rametros
   necesarios
14 # para iniciar la simulaci n de MMU.
```

Capítulo 4

Conclusiones

1. **Elegir bien el algoritmo de reemplazo de páginas cambia por completo el rendimiento del sistema.** Cuando haces una simulación, se nota enseguida cómo cambia la historia según el algoritmo que uses, aunque las tareas sean exactamente las mismas. Al mirar datos como el total de fallos de página, cuánto tarda en completarse todo o la memoria que se consume, queda clarísimo que la forma de elegir qué página sacrificar afecta un montón a la eficiencia. Esto nos demuestra que no da igual usar cualquiera; hay que elegir el adecuado según el tipo de trabajo que tenga el sistema (Silberschatz et al., 2018).
2. **Dividir el software en partes independientes (modularidad) hace que sea más fácil mejorarlo y mantenerlo.** Al separar bien las tareas, dejando por un lado la interfaz, por otro la creación de instrucciones, la simulación en sí, la gestión de memoria y los gráficos, puedes meterle mano a un componente sin miedo a romper el resto. Gracias a esta estructura, si mañana quieres añadir un algoritmo nuevo, otra métrica o cualquier función extra, puedes hacerlo sin tener que poner patas arriba todo el código. El proyecto se vuelve mucho más escalable y fácil de cuidar.
3. **Simular el proceso es la mejor manera de entender de verdad cómo funciona la memoria virtual.** Ver en una pantalla qué está pasando con la RAM, la memoria virtual, las páginas y los procesos ayuda a que la teoría aburrida de los sistemas operativos se convierta en algo real que puedes ver con tus propios ojos. Así es mucho más fácil pillar por qué ocurren los fallos de página, cómo se decide qué se borra y en qué se diferencian los algoritmos (Tanenbaum & Bos, 2015). Al final, la simulación es una herramienta tremenda tanto para estudiar como para ponerse a experimentar.

Bibliografía

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10.^a ed.). Wiley. <https://os.ecci.ucr.ac.cr/slides/Abraham-Silberschatz-Operating-System-Concepts-10th-2018.pdf>
- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4.^a ed.). Pearson. <https://os.ecci.ucr.ac.cr/slides/Andrew-S.-Tanenbaum-Modern-Operating-Systems.pdf>